

What is my sql:-

MySQL is an open-source relational database management system (RDBMS) that uses SQL to store, manage, and retrieve data in a structured way.

It stores data in tables with rows and columns and supports relationships between tables using primary and foreign keys. MySQL is fast, secure, and scalable, so it is widely used in web applications with Python, Django, Java, and PHP. It also supports transactions, indexing, joins, views, and stored procedures, which help in building reliable and high-performance applications.

MySQL is a software that works as a Relational Database Management System (RDBMS). It is used to store data in table format and is managed using the SQL language.

What is DBMS?

DBMS (Database Management System) is software that allows users to store, manage, update, and retrieve data from a database efficiently and securely. It acts as an interface between the user and the database.

What is a Database?

A database is an organized collection of data that is stored electronically and can be easily accessed, managed, and updated.

Types of Databases

1. Relational Database

- Stores data in tables
- Example: MySQL, Oracle, PostgreSQL

2. Non-Relational (NoSQL) Database

- Stores data in documents, key-value, columns, or graphs
- Example: MongoDB, Redis

3. Hierarchical Database

- Data stored in tree structure
- Example: IBM IMS

4. Network Database

- Data stored as graph links
- Example: IDMS

5. Object-Oriented Database

- Data stored as objects
- Example: ObjectDB

What is RDBMS?

RDBMS (Relational Database Management System) is a type of database system that stores data in the form of tables (rows and columns) and maintains relationships between those tables using keys like Primary Key and Foreign Key. It ensures data accuracy, consistency, and avoids duplication.

What is Non-RDBMS?

Non-RDBMS (also called NoSQL) is a type of database that does not store data in tables and does not use fixed schemas like RDBMS. It is designed to handle large amounts of unstructured or semi-structured data.

DBMS	RDBMS
Stores data as files	Stores data in tables (rows & columns)
Does not support relationships	Supports relationships using keys
No Primary Key	Uses Primary Key
More data redundancy	Less data redundancy
Less secure	More secure
Example: File system	Example: MySQL, Oracle

What is data?

Data is raw facts or information that can be stored and processed.

What is data integrity?

It refers to checking the correctness and consistency of data.

What is Data Consistency?

Data consistency means that the same data has the same value everywhere in the database.

Example

If a user's balance is ₹1000 in one table and ₹500 in another → not consistent

If both show ₹1000 → consistent

What is a Server?

A server is a computer that stores the database and processes user requests.

difference between data integrity and data consistency ?

Data Integrity	Data Consistency
Means data is correct and valid	Means data is same everywhere
Prevents wrong or fake data	Prevents mismatch of values
Uses constraints (PK, FK, NOT NULL)	Uses transactions & rules
Example: Age cannot be negative	Example: Balance must be same in all tables

What is SQL?

SQL (Structured Query Language) is a standard language used to communicate with a database to store, retrieve, update, and delete data.

Difference between SQL and MySQL

SQL	MySQL
A language	A database software
Used to write queries	Uses SQL to manage data

Not a DB	Is an RDBMS
----------	-------------

What is a Query?

A query is a request to the database to get, insert, update, or delete data. A piece of code line which can be used to communicate the database to store or review the data from the database.

Example

```
SELECT * FROM students;
```

What is a Subquery?

A subquery is a query written inside another query. It is used to get data that the main query depends on.

Example

```
SELECT name  
FROM students  
WHERE marks > (SELECT AVG(marks) FROM students);
```

What is a View?

A view is a virtual table created from a SELECT query. It does not store data itself, but shows data from one or more tables.

Example

```
CREATE VIEW high_marks AS  
SELECT name, marks FROM students WHERE marks > 80;
```

What is a Table?

A table is a structure in a database where data is stored in rows and columns.

What is a Record?

A record is a single row in a table that contains complete information about one item.

What is a Field / Column?

A field (column) is a single piece of data in a table. It represents one type of information.

What is a Primary Key?

A primary key is a column that uniquely identifies each record in a table.

It cannot be NULL or duplicated.

What is a Foreign Key?

A foreign key is a column that links one table to the primary key of another table.

It maintains the relationship between tables.

What is NULL?

NULL means no value or unknown value in a column.

It is not the same as 0 or an empty string.

What is an Index?

An index is a database object that makes data search faster. It works like an index in a book.

Workbench guidelines about safe updates:-

What is Safe Update Mode in SQL?

Safe Update Mode is a MySQL safety feature that prevents you from:

- Updating all rows by mistake
- Deleting all rows by mistake

It blocks UPDATE and DELETE queries that:

- Do NOT use a WHERE condition
- Or do NOT use a KEY column in the WHERE

Turn off safe mode (temporary)

```
SET SQL_SAFE_UPDATES = 0;
```

Then run your UPDATE again.

To turn it back ON:

```
SET SQL_SAFE_UPDATES = 1;
```

SQL commands

◆ DDL – Data Definition Language

Used to create and change database structure.

- CREATE – create table/database
 - ALTER – change table
 - DROP – delete table/database
 - TRUNCATE – remove all records
-

◆ DML – Data Manipulation Language

Used to work with data.

- INSERT – add data
 - UPDATE – change data
 - DELETE – remove data
-

◆ DQL – Data Query Language

Used to fetch data.

- SELECT – get data
-

◆ DCL – Data Control Language

Used for permissions.

- GRANT – give access
 - REVOKE – remove access
-

◆ TCL – Transaction Control Language

Used to manage transactions.

- COMMIT
 - ROLLBACK
 - SAVEPOINT
-

◆ DDL – Data Definition Language

Used to create and change database structure.

CREATE: – create table/database it can be used for create a table structure in database by using .

Example:-

```
mysql> create table siva(sino int,name varchar(20),mobile_number  
varchar(15));
```

ALTER –

change table it can be used fro change the structure of the table like ALTER is used to add, delete, or modify columns in a table.

>it can be used for modifying structure of the table outside

Examples:-

```
mysql> alter table siva add column branch varchar(20);  
mysql> alter table siva drop branch;  
mysql> alter table siva modify branch int;
```

DROP: –

DROP is used to permanently delete a database object like a table or database.

```
mysql> drop table siva;  
mysql> drop database siva;
```

DROP	TRUNCATE
Deletes table structure + data	Deletes only data
Cannot be rolled back	Cannot be rolled back
Table is removed completely	Table remains

TRUNCATE :-

TRUNCATE is used to delete all records from a table very fast, but it keeps the table structure.

```
mysql> truncate table siva;
```

DML – Data Manipulation Language

Used to work with data.

INSERT: –

It can be used to inserting the data into tables

They are two ways to inserting the data

```
mysql> insert into siva values(1,"siva","6300157088");
```

(or)

```
mysql> insert into siva  
values(2,"chinna","63001524"),(3,"satya","5566983");
```

```
output:- mysql> SELECT* FROM SIVA;
```

```
+-----+-----+-----+  
| sino | name  | mobile_number |  
+-----+-----+-----+  
| 1 | siva  | 6300157088    |  
| 2 | chinna | 63001524      |  
| 3 | satya  | 5566983       |  
+-----+-----+-----+  
3 rows in set (0.03 sec)
```

UPDATE: –

It can be used for set are updateing the information inside of the table.

```
mysql> alter siva add column email varhar(8);
```

```
mysql> select* from siva;
```

```
+-----+-----+-----+-----+  
| sino | name  | mobile_number | email |  
+-----+-----+-----+-----+  
| 1 | siva  | 6300157088    | NULL |  
| 2 | chinna | 63001524      | NULL |  
| 3 | satya  | 5566983       | NULL |  
| 4 | naveen | 69856412      | NULL |  
| 5 | ghty   | 6302960441    | NULL |  
+-----+-----+-----+-----+  
5 rows in set (0.00 sec)
```



```
mysql> update siva set email="nsiva@gmail.com" where sino=1;
```

(or)

```
mysql> update siva set email=case when sino=1 then "siva@gmail"
when sino=2 then "chinna@gmail.com" when sino=3 then
"satya@gmail.com" when sino=4 then "naveen@gmail" when
sino=5 then "ghty@gmail..com" else email end;
```

```
mysql> select* from siva;
```

```
+-----+-----+-----+-----+
| sino | name  | mobile_number | email          |
+-----+-----+-----+-----+
| 1 | siva  | 6300157088   | siva@gmail    |
| 2 | chinna | 63001524     | chinna@gmail.com |
| 3 | satya  | 5566983      | satya@gmail.com |
| 4 | naveen | 69856412     | naveen@gmail   |
| 5 | ghty   | 6302960441   | ghty@gmail..com |
+-----+-----+-----+-----+
5 rows in set (0.00 sec)
```

What is CASE in SQL?

CASE is a conditional statement in SQL. It works like an IF–ELSE statement to apply different values based on conditions.

DELETE :– remove data

DELETE is used to remove records (rows) from a table.

Examples:-

Delete one row

```
DELETE FROM siva
```

```
WHERE sino = 1;
```

Delete all rows

```
DELETE FROM siva;
```

Difference: DELETE vs TRUNCATE

DELETE	TRUNCATE
Can use WHERE	Cannot use WHERE
Can rollback	Cannot rollback
Slower	Faster

DCL – Data Control Language

Used for permissions.

GRANT :- The **GRANT** command is used to **give permissions (privileges)** to database users or roles so they can access or modify database objects like:

- Tables
- Views
- Sequences
- Procedures
- Schemas

It is part of **DCL (Data Control Language)**.

General Syntax

```
GRANT privilege_list  
ON object_name  
TO user_or_role;
```

Part

Description

GRANT

SQL keyword

privilege_list

What action is allowed (SELECT, INSERT, etc.)

ON object_name

Which table/view/etc.

TO user_or_role

Who gets permission

Common Privileges

Privilege	Meaning
SELECT	Read data
INSERT	Add new rows
UPDATE	Modify rows
DELETE	Remove rows
ALL	All permissions
REFERENCES	Create foreign key
EXECUTE	Run procedures
ALTER	Change structure

Task	Command
All users	SELECT user, host FROM mysql.user;
Current user	SELECT USER();
User permissions	SHOW GRANTS FOR 'user'@'host';
Active users	SHOW PROCESSLIST;

How to see current user?

```
mysql> select user();
+-----+
| user() |
+-----+
| root@localhost |
+-----+
1 row in set (0.01 sec)
```

Display all user in your sql?

```
mysql> SELECT user, host FROM mysql.user;
+-----+-----+
| user | host |
```

```
+-----+-----+
| mysql.infoschema | localhost |
| mysql.session    | localhost |
| mysql.sys        | localhost |
| root             | localhost |
+-----+-----+
5 rows in set (0.00 sec)
```

Create user in your host?

```
mysql> create user 'siva'@'localhost' identified by "Siva@6300";
mysql> select user,@localhost from mysql.user;
```

```
+-----+-----+
| user          | @localhost      |
+-----+-----+
| mysql.infoschema | NULL           |
| mysql.session    | NULL           |
| mysql.sys        | NULL           |
| root            | NULL           |
| siva            | NULL           |
```

Give permission to user?

1. Grant SELECT permission

GRANT SELECT ON employees TO siva;

➡ siva can only read the employees table.

2. Grant multiple permissions

GRANT SELECT, INSERT, UPDATE ON employees TO siva;

➡ siva can read, add, and update records.

3. Grant ALL permissions

GRANT ALL ON employees TO siva;

➡ Full access to the table.

4. Grant to Multiple Users

GRANT SELECT ON employees TO siva, ramesh;

5. Grant with WITH GRANT OPTION

Allows the user to give permissions to others.

GRANT SELECT ON employees TO siva WITH GRANT OPTION;

➡ Now siva can give SELECT permission to others.

6. Grant on Database Level (MySQL)

GRANT ALL PRIVILEGES ON companydb.* TO siva;

7. Grant on Specific Columns (MySQL)

GRANT UPDATE (salary) ON employees TO siva;

➡ siva can update only the **salary** column.

8.drop the user?

It can be used for drop or delete the user from the server

Drop user "siva"@localhost";

Here @ is a separator it can indicate the user name and localhost for clear understanding

9.showing grants ?

show grants for "siva"@localhost"

🔄 Check Permissions

SHOW GRANTS FOR siva;

Important Notes

- Only **DBA/admin** can use GRANT.
- GRANT changes are **immediate**.
- Some DBs require FLUSH PRIVILEGES; (MySQL).

Give all access to user :-

To **give back full access on all databases** to that user, use this GRANT query 🙌

```
GRANT ALL PRIVILEGES
ON *.*
TO 'siva'@'localhost'
WITH GRANT OPTION;
```

FLUSH PRIVILEGES = Refresh user permissions in MySQL.

REVOKE :- REVOKE is used to take back (remove) permissions from a user

Basic Syntax:-

```
REVOKE privilege_name
ON database_name.table_name
FROM 'username'@'host';
```

Revoke all privileges:-

```
REVOKE ALL PRIVILEGES
ON *.*
FROM 'siva'@'localhost';
```

Part	Meaning
First *	All databases
.	Separator
Second *	All tables in all databases

	GRANT	REVOKE
Feature		
Purpose	Give permissions	Remove permissions
Meaning	Allow user to do something	Stop user from doing something
	GRANT SELECT ON db.table TO 'siva'@'localhost';	REVOKE SELECT ON db.table FROM 'siva'@'localhost';
Effect	User can access	User loses access

For practice purposes :-

```
show databases
select user()
select user,host from mysql.user
create user "siva"@"localhost" identified by "siva1243";
select user,host from mysql.user
drop user "siva"@"localhost";
drop user "ms"@"localhost"
select user,host from mysql.user;
create user "siva"@"localhost" identified by "siva1243"
select user,host from mysql.user
drop user "siva"@"localhost"
create user "siva"@"localhost" identified by "145"
use rishi;
grant select on employ to "siva"@"localhost"
select user,host from mysql.user
show grants for "siva"@"localhost"
revoke all privileges on *.* from "siva"@"localhost";
drop user "siva"@"localhost"
create user "siva"@"localhost" identified by "siva";
select user,host from mysql.user;
grant all privileges on *.* to "siva"@"localhost";
show grants for "siva"@"localhost"
revoke all privileges on *.* from "siva"@"localhost";
show grants for "siva"@"localhost";
select user,host from mysql.user;
drop user "siva"@"localhost";
```

Transaction Control Language

Used to manage transactions.

COMMIT:-

This can be used to permanently save all changes made during the current transaction to the database
Confirm & save changes permanently.

Before commit this data we have

```
show databases;  
use rishi;  
show tables;  
select* from employ;  
insert into employ values(9,"amma",25000),(10,"nana",500000);  
commit;
```

Sino	name	salary
1	Alice	55000.00
2	Bob	62000.00
3	Charlie	48000.00
4	Diana	70000.00
5	Ethan	53000.00
6	siva	2500.00
7	jkh	2000.00
8	satya	1524.00
9	amma	25000.00
10	nana	500000.0

After commit I can insert another data in that table

```
insert into employ values(10,"sister",45000),(11,"brother",45000);  
select* from employ;
```


We have this data:-

1	Alice	55000.00
2	Bob	62000.00
3	Charlie	48000.00
4	Diana	70000.00
5	Ethan	53000.00
6	siva	2500.00
7	jkh	2000.00
8	satya	1524.00
9	amma	25000.00
10	nana	500000.00
11	sister	45000.00
12	brother	45000.00

ROLLBACK:-

This can be used for undo all changes made during the current transaction restoring the databases to exact state where you commit in the transaction .

```
rollback;  
select* from employ;
```

1	Alice	55000.00
2	Bob	62000.00
3	Charlie	48000.00
4	Diana	70000.00
5	Ethan	53000.00
6	siva	2500.00
7	jkh	2000.00
8	satya	1524.00
9	amma	25000.00
10	nana	500000.0

SAVEPOINT

SAVEPOINT is a marker (checkpoint) inside a database transaction that allows you to roll back part of the transaction without canceling the entire transaction.

What is a Clause in SQL?

A clause is a keyword in an SQL query that tells the database what to do with data.

SQL Clause Execution Order

FROM
WHERE
GROUP BY
HAVING
SELECT
ORDER BY
LIMIT/offset

What is LIMIT?

LIMIT tells SQL how many rows to show.

SELECT * FROM employ

LIMIT 5;

Meaning:

“Show only the first 5 rows.”

What is OFFSET?

OFFSET tells SQL how many rows to skip before showing.

SELECT * FROM employ

LIMIT 5 OFFSET 5;

Meaning:

“Skip first 5 rows, then show next 5.”

4. CONDITION Clauses

Used to apply **conditions**.

Clause	Purpose
AND	All conditions true
OR	Any condition true
NOT	Reverse condition

Clause	Purpose
IN	Match many values
BETWEEN	Range
LIKE	Pattern search
EXISTS	Check subquery

1 AND – All conditions must be true

```
SELECT * FROM employ
WHERE dept = 'cse' AND salary > 40000;
```

→ Only CSE employees with salary > 40000.

2 OR – Any one condition true

```
SELECT * FROM employ
WHERE dept = 'cse' OR dept = 'ece';
```

→ All CSE and ECE employees.

3 NOT – Reverse condition

```
SELECT * FROM employ
WHERE NOT dept = 'cse';
```

→ All employees except CSE.

4 IN – Match many values

```
SELECT * FROM employ
WHERE dept IN ('cse', 'mec', 'civil');
```

5 BETWEEN – Range (inclusive)

```
SELECT * FROM employ
WHERE salary BETWEEN 30000 AND 70000;
```

6 LIKE – Pattern search

```
SELECT * FROM employ
WHERE name LIKE 'A%';
```

→ Names starting with A.
Other patterns:

- %a → ends with a
- %a% → contains a
- _a% → second letter a

7 EXISTS – Check subquery

```
SELECT name
FROM employ e
WHERE EXISTS (
    SELECT 1
    FROM employ
    WHERE dept = 'cse' AND salary > 50000
);
```

→ Returns all names if at least one CSE employee earns > 50000.

Constraints in sql:-

Constraints are rules applied to table columns to control the type of data that can be stored in a table.

Constraint	Purpose
NOT NULL	No empty values
UNIQUE	No duplicate values
PRIMARY KEY	Unique row identifier
FOREIGN KEY	Links two tables
CHECK	Condition-based values
DEFAULT	Automatic value
AUTO_INCREMENT	Auto numbering

NOT NULL:-

NOT NULL is a constraint in MySQL that does not allow a column to store NULL (empty) values.

```
CREATE TABLE student (
    name VARCHAR(50) NOT NULL
);
```

UNIQUE:-

UNIQUE is a constraint in MySQL that **does not allow duplicate values** in a column.

```
CREATE TABLE student (  
    email VARCHAR(100) UNIQUE  
);
```

PRIMARY KEY:-

A **PRIMARY KEY** is a constraint that uniquely identifies each record (row) in a table.

It is a combination of two rules:

- **UNIQUE** → no duplicate values
- **NOT NULL** → no empty values

So, a primary key value is always unique and always present.

Why PRIMARY KEY is important

- Identifies each row **uniquely**
- Helps in **fast searching** (indexing)
- Used to create **relationships** with other tables (FOREIGN KEY)
- Ensures **data integrity**

```
CREATE TABLE student (  
    id INT PRIMARY KEY,  
    name VARCHAR(50)  
);
```

```
CREATE TABLE marks (  
    student_id INT,  
    subject_id INT,  
    PRIMARY KEY (student_id,  
    subject_id)  
);
```

FOREIGN KEY:-

A **FOREIGN KEY** is a constraint used to **create a relationship between two tables**.

Parent Table

```
CREATE TABLE department (  
    dept_id INT PRIMARY KEY,  
    dept_name VARCHAR(50)  
);
```

Child Table

```
CREATE TABLE employee (  
    emp_id INT PRIMARY KEY,  
    emp_name VARCHAR(50),  
    dept_id INT,  
    FOREIGN KEY (dept_id)  
    REFERENCES department(dept_id)  
);
```

CHECK:-

The CHECK constraint is used to limit the values that can be stored in a column by applying a condition (rule).

- ☐ CHECK validates data **before inserting or updating**
- ☐ If condition fails → **error occurs**

```
CREATE TABLE employee (  
    emp_id INT PRIMARY KEY,  
    salary INT CHECK (salary > 0),  
    age INT CHECK (age >= 21)  
);
```

DEFAULT:-

The **DEFAULT** constraint is used to **assign a predefined value** to a column **when no value is provided** during insertion.

```
CREATE TABLE student (  
    id INT PRIMARY KEY,  
    status VARCHAR(20) DEFAULT 'active'  
);
```

AUTO INCREMENT:-

AUTO_INCREMENT is a MySQL attribute used to automatically generate unique numbers for a column, usually for PRIMARY KEY values.

👉 It increases the value automatically whenever a new row is inserted.

✂ Why AUTO_INCREMENT is used

- Avoids manual ID entry
- Ensures uniqueness
- Commonly used for ID columns

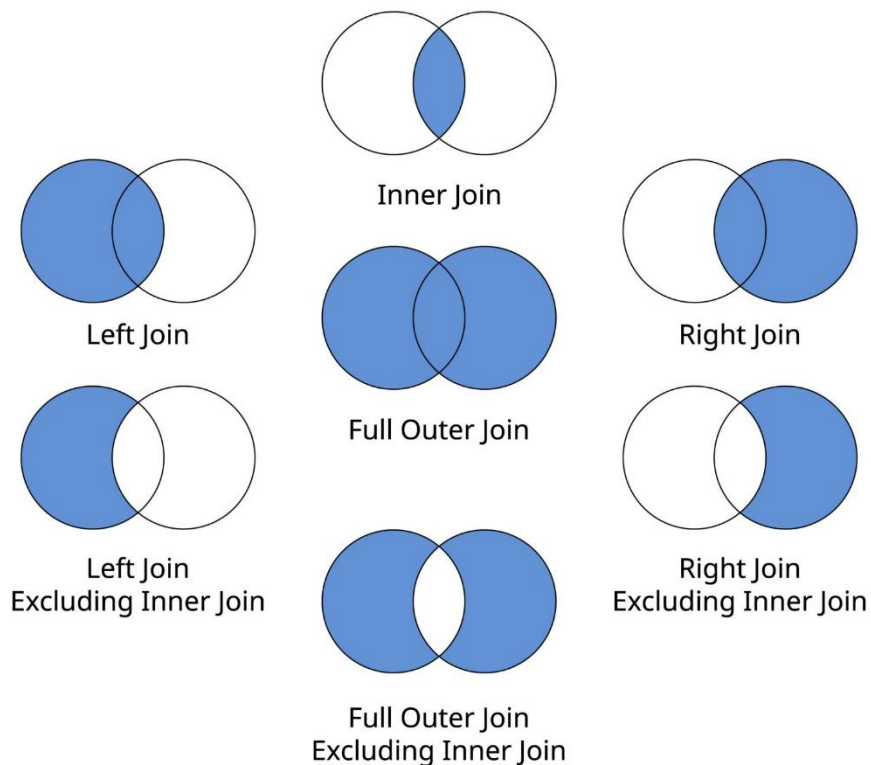
```
CREATE TABLE student (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  name VARCHAR(50)  
);
```

OUTPUT:-

id	name
1	Siva
2	Ravi

```
INSERT INTO student (name) VALUES  
( 'Siva');  
INSERT INTO student (name) VALUES  
( 'Ravi');
```

SQL JOINS



Inner join:-

- It is one of the type in joins
- It can be used to joining two tables
- It can be returning only matched rows only from both tables
- It can be used primary key and foreign key from tables based on this constraints this can be filters the data from table and then gives the output.

Syntax:-

```
SELECT columns  
FROM table1  
JOIN table2  
ON table1.column = table2.column;
```

Example:-

```
mysql> create table a(id int primary key unique,name varchar(20));  
Query OK, 0 rows affected (0.15 sec)
```

```
mysql> insert into a values(1,"siva"),  
-> (2,"satya"),  
-> (3,"amma"),  
-> (4,"nana"),  
-> (5,"sisters");  
Query OK, 5 rows affected (0.01 sec)  
Records: 5 Duplicates: 0 Warnings: 0
```

```
mysql> select* from a;  
+----+-----+  
| id | name  |  
+----+-----+  
| 1  | siva  |  
| 2  | satya |  
| 3  | amma  |  
| 4  | nana  |  
| 5  | sisters|  
+----+-----+  
5 rows in set (0.00 sec)
```

```
mysql> create table b (si_no int primary key,marks int);  
Query OK, 0 rows affected (0.04 sec)  
mysql> insert into b values(1,20),  
-> (2,50),
```

```
-> (3,55),
-> (4,456),
-> (11,256),
-> (10,25),
-> (0,0);
```

Query OK, 7 rows affected (0.01 sec)

Records: 7 Duplicates: 0 Warnings: 0

```
mysql> select* from b;
```

```
+-----+-----+
| si_no | marks |
+-----+-----+
| 0 | 0 |
| 1 | 20 |
| 2 | 50 |
| 3 | 55 |
| 4 | 456 |
| 10 | 25 |
| 11 | 256 |
+-----+-----+
```

7 rows in set (0.00 sec)

```
mysql> select* from a inner join b on b.si_no=a.id;
```

```
+---+-----+-----+-----+
| id | name | si_no | marks |
+---+-----+-----+-----+
| 1 | siva | 1 | 20 |
| 2 | satya | 2 | 50 |
| 3 | amma | 3 | 55 |
| 4 | nana | 4 | 456 |
+---+-----+-----+-----+
```

4 rows in set (0.03 sec)

Left-join:-

- It can be used for joining the two tables
- It can return matched rows from right table and all records from left table is called left joining
- If in case left row does not match to right table row then it can be return that row but it can be written null keyword

Example:-

```
mysql> select b.si_no,b.marks,a.name from a left join b on
b.si_no=a.id;
+-----+-----+-----+
| si_no | marks | name   |
+-----+-----+-----+
| 1     | 20    | siva   |
| 2     | 50    | satya  |
| 3     | 55    | amma   |
| 4     | 456   | nana   |
| NULL  | NULL  | sisters|
+-----+-----+-----+
5 rows in set (0.01 sec)
```

Right -join:-

- It can be used for joining the two tables
- It can return matched rows from left table and all records from right table is called right joining
- If in case right table row does not match to left table row then it can be return that row but it can be written null keyword

Example:-

```
mysql> select a.id,a.name,b.marks from a right join b on a.id=b.si_no;
```

```
+-----+-----+-----+
| id  | name  | marks |
+-----+-----+-----+
| NULL | NULL  | 0     |
| 1    | siva  | 20    |
| 2    | satya | 50    |
| 3    | amma  | 55    |
| 4    | nana  | 456   |
| NULL | NULL  | 25    |
| NULL | NULL  | 256   |
+-----+-----+-----+
```

7 rows in set (0.00 sec)

FULL JOIN (FULL OUTER JOIN)

Definition:

FULL JOIN returns all records from both tables — matching rows + non-matching rows from both sides.

If there is no match:

- Left table columns → data, Right table → NULL
- Right table columns → data, Left table → NULL

```
mysql> select* from a left join b on b.si_no=a.id
-> union
-> select* from a right join b on b.si_no =a.id;
```

```
+-----+-----+-----+-----+
| id  | name  | si_no | marks |
+-----+-----+-----+-----+
| 1    | siva  | 1     | 20    |
| 2    | satya | 2     | 50    |
| 3    | amma  | 3     | 55    |
| 4    | nana  | 4     | 456   |
| 5    | sisters | NULL  | NULL  |
| NULL | NULL  | 0     | 0     |
| NULL | NULL  | 10    | 25    |
| NULL | NULL  | 11    | 256   |
+-----+-----+-----+-----+
```

CROSS JOIN

Definition:

CROSS JOIN returns the Cartesian product of two tables — every row of the first table is combined with every row of the second table

Important Points

1. No matching condition is required.
2. Result rows = rows of table1 $\times$ rows of table2.
3. Can generate **huge data** if tables are large.
4. Also called **Cartesian Join**.

```
mysql> insert into color values("yellow"),("red"),("green");
Query OK, 3 rows affected (0.01 sec)
Records: 3 Duplicates: 0 Warnings: 0
```

```
mysql> select* from color;
```

```
+-----+
| name |
+-----+
| yellow |
| red |
| green |
+-----+
```

```
mysql> create table sizes( name varchar(20));
Query OK, 0 rows affected (0.05 sec)
```

```
mysql> insert into sizes values("m"),("s"),("l");
Query OK, 3 rows affected (0.01 sec)
Records: 3 Duplicates: 0 Warnings: 0
```

```
mysql> select* from colors cross join sizes;
ERROR 1146 (42S02): Table 'siva.colors' doesn't exist
mysql> select* from color cross join sizes;
```

```
+-----+-----+
| name | name |
+-----+-----+
| green | m |
| red | m |
| yellow | m |
| green | s |
| red | s |
| yellow | s |
| green | l |
| red | l |
| yellow | l |
+-----+-----+
```

```
9 rows in set (0.01 sec)
```

SELF JOIN

Definition:

SELF JOIN is a join where a table is joined with itself.

It is used when data in the same table is related to other data in the same table.

Simple Example (Employee & Manager)

Table: employees

emp_id	emp_name	manager_id
1	Siva	NULL
2	Ravi	1
3	Priya	1
4	Kiran	2

Here:

- manager_id refers to emp_id in the same table

SELF JOIN Query

```
SELECT e.emp_name AS employee,  
       m.emp_name AS manager  
FROM employees e  
JOIN employees m  
ON e.manager_id = m.emp_id;
```

Result

employee manager

Ravi	Siva
Priya	Siva
Kiran	Ravi

Example:-

```
mysql> update a set role_id=case when id=1 then 1 when id=2 then 1 when id=3 then 3 when id=4 then 2 when id=3 then 4  
when id=5 then 4 else role_id end;  
Query OK, 4 rows affected (0.01 sec)  
Rows matched: 5 Changed: 4 Warnings: 0  
  
mysql> select* from a;  
+----+-----+-----+  
| id | name  | role_id |  
+----+-----+-----+
```

```

| 1 | siva | 1 |
| 2 | satya | 1 |
| 3 | amma | 3 |
| 4 | nana | 2 |
| 5 | sisters | 4 |
+-----+-----+
5 rows in set (0.00 sec)

mysql> select* from a left join a on a.id=a.role_id;
ERROR 1066 (42000): Not unique table/alias: 'a'
mysql> select * from a as id left join a as role on id.id=role.role_id;
+-----+-----+-----+-----+
| id | name | role_id | id | name | role_id |
+-----+-----+-----+-----+
| 1 | siva | 1 | 2 | satya | 1 |
| 1 | siva | 1 | 1 | siva | 1 |
| 2 | satya | 1 | 4 | nana | 2 |
| 3 | amma | 3 | 3 | amma | 3 |
| 4 | nana | 2 | 5 | sisters | 4 |
| 5 | sisters | 4 | NULL | NULL | NULL |
+-----+-----+-----+-----+
6 rows in set (0.02 sec)

mysql>

```

>you must give alias name to table left table and right table alias name without giving any alias name it can be throws the errors.

What is string functions:-

String functions are built-in SQL functions used to manipulate, modify, extract, and format text (character) data.

1. Case Conversion Functions

Used to change letter case.

Function Example

UPPER() UPPER('sql') → SQL

LOWER() LOWER('SQL') → sql

2. Concatenation Functions

Used to join strings.

Function Example

CONCAT() CONCAT('Hello',' ','World')

CONCAT_WS() CONCAT_WS('-', '2026','01','29')

3. Substring / Extraction Functions

Used to extract part of string.

Function	Example
SUBSTRING()	SUBSTRING('Database',1,4) → Data
LEFT()	LEFT('Python',3) → Pyt
RIGHT()	RIGHT('Python',2) → on

4. Length Functions

Used to find length.

Function	Example
LENGTH()	LENGTH('Siva') → 4
CHAR_LENGTH()	CHAR_LENGTH('😊') → 1

5. Search Functions

Used to find characters.

Function	Example
INSTR()	INSTR('Hello','e') → 2
LOCATE()	LOCATE('lo','Hello') → 4

6. Replace & Trim Functions

Used for cleaning data.

Function	Example
REPLACE()	REPLACE('Java','a','o')
TRIM()	TRIM(' siva ')
LTRIM()	LTRIM(' hi')
RTRIM()	RTRIM('hi ')

Real-Time Example

Table: users

id email

1 siva@gmail.com

Extract username:

SELECT SUBSTRING_INDEX(email,'@',1) FROM users;

Date functions:-

Date functions are built-in SQL functions used to work with date and time values, such as getting the current date, extracting parts of a date, adding or subtracting days, and formatting dates.

They work on data types:

- DATE
- DATETIME
-

Type of Date Function	Purpose	Function	Code Example
Current Date & Time	Get current date/time	CURDATE()	SELECT CURDATE();
		NOW()	SELECT NOW();
		CURRENT_TIMESTAMP()	SELECT CURRENT_TIMESTAMP();
Date Part Extraction	Get day, month, year	DAY()	SELECT DAY('2024-08-15');
		MONTH()	SELECT MONTH('2024-08-15');
		YEAR()	SELECT YEAR('2024-08-15');
Date Calculation	Add/subtract date	DATE_ADD()	SELECT DATE_ADD('2024-01-01', INTERVAL 10 DAY);
		DATE_SUB()	SELECT DATE_SUB('2024-01-01', INTERVAL 1 MONTH);
Date Difference	Find difference	DATEDIFF()	SELECT DATEDIFF('2024-12-31','2024-01-01');

		TIMESTAMPDIFF()	SELECT TIMESTAMPDIFF(YEAR,'2000-01-01',CURDATE());
Date Formatting	Format date	DATE_FORMAT()	SELECT DATE_FORMAT(CURDATE(),'%d-%m-%Y');
Month End	Last day of month	LAST_DAY()	SELECT LAST_DAY('2024-02-10');

Stored Procedures

A stored procedure is a precompiled collection of SQL statements that can be executed by calling its name.

Why we use Stored Procedures

- Reuse code (write once, use many times)
- Faster execution (precompiled)
- Better security (users call procedure, not tables)
- Reduces network traffic
- Easy maintenance

Basic Syntax (MySQL)

Create a stored procedure

```
DELIMITER //
```

```
CREATE PROCEDURE getAllStudents()
BEGIN
    SELECT * FROM students;
END //
```

```
DELIMITER ;
```

Call the procedure:-

```
CALL getAllStudents();
```

